

Push_swap

Porque Swap_push não soa tão natural

Resumo:

Esta atividade vai fazer você ordenar dados em uma stack, com um conjunto limitado de instruções, usando o menor número possível de ações. Para ter sucesso, você terá que manipular vários tipos de algoritmos e escolher a solução mais apropriada (dentre várias) para uma ordenação de dados otimizada.

Esta é uma atividade em grupo, a ser realizada por exatamente 2 learners.

Versão: 1.1 (tradução)

Sumário

- I. Prefácio**
- II. Instruções Comuns**
- III. Instruções sobre IA**
- IV. Introdução**
- V. Objetivos**
- VI. Parte Obrigatória**
 - VI.1 Requisitos do projeto em grupo
 - VI.2 As regras
 - VI.3 Requisitos do algoritmo
 - VI.4 Exemplo
 - VI.5 O programa "push_swap"
 - VI.6 Benchmark de Performance
- VII. Requisitos do Readme**
- VIII. Parte Bônus**
 - VIII.1 O programa "checker"
- IX. Submissão e avaliação entre pares**

CAPÍTULO I

Prefácio

Era uma vez, nas terras misteriosas da **Ciência da Computação**, um certo Donald Knuth¹ popularizou a notação *Big-O* para nos ajudar a falar sobre como algoritmos escalam. Big-O é basicamente uma forma educada de dizer: *"Seu código ou vai rodar rápido para sempre... ou vai ficar tão lento que você terá tempo de fazer um café, tomá-lo, e envelhecer esperando."*

Reza a lenda que, nos primeiros tempos, os programadores não tinham Big-O. Eles simplesmente rodavam o código e, se demorasse demais, culpavam o *hardware*. Então o Big-O chegou e estragou a diversão ao provar matematicamente que às vezes o seu algoritmo é simplesmente **ruim** — não importa quantos hamsters você coloque na roda da CPU.

Por que isso importa para o push_swap? Porque aqui está você, armado com duas stacks e um punhado de movimentos estranhamente limitados, tentando ordenar números mais rápido que um insertion sort sonolento. O Big-O vai te ajudar a encarar a verdade brutal: uma estratégia brilhante $O(n \log n)$ sempre vai superar sua estratégia desajeitada $O(n^2)$ conforme a entrada cresce... a menos que, é claro, você escolha ignorar a matemática e assistir sua contagem de operações disparar.

Então, ao desenhar seus algoritmos, lembre-se: o Big-O não está aqui para te assustar — está aqui para evitar que você se torne a pessoa que escreve:

```
pb; pa; pb; pa; pb; pa; ...
```

10.000 vezes seguidas, e depois se pergunta por que o checker os odeia. **Ordene com inteligência, não com pressa.**

Big-O não é uma classificação de tamanho de pipoca... mas se fosse, $O(n \log n)$ ainda seria o ponto ideal.

¹ Sim, o cara da barba gloriosa e paciência infinita.

CAPÍTULO II

Instruções Comuns

- Seu projeto deve ser escrito em **C**.
- Seu projeto deve ser escrito de acordo com a **Norma**. Se você tiver arquivos/funções bônus, eles serão incluídos na checagem da Norma, e você receberá **0** se houver erro de Norma.
- Suas funções não devem encerrar inesperadamente (segmentation fault, bus error, double free, etc.), exceto por comportamento indefinido. Se isso ocorrer, seu projeto será considerado não funcional e receberá **0** durante a avaliação.
- Toda memória alocada no heap deve ser devidamente liberada quando necessário. Vazamentos de memória não serão tolerados.
- Se o subject exigir, você deve entregar um **Makefile** que compila seus arquivos fonte para o output exigido com as flags `-Wall`, `-Wextra` e `-Werror`, usando `cc`. Além disso, seu Makefile não deve realizar relink desnecessário.
- Seu Makefile deve conter, no mínimo, as regras `$(NAME)`, `all`, `clean`, `fclean` e `re`.
- Para submeter bônus do seu projeto, você deve incluir uma regra `bonus` no seu Makefile, que adicionará todos os headers, bibliotecas ou funções diversas que não são permitidos na parte principal do projeto. Os bônus devem ser colocados em arquivos `_bonus.{c/h}`, a menos que o subject especifique o contrário. A avaliação das partes obrigatória e bônus é feita separadamente.
- Se seu projeto permitir o uso da sua libft, você deve copiar suas fontes e o Makefile associado para uma pasta `libft`. O Makefile do seu projeto deve compilar a biblioteca usando o Makefile dela, e então compilar o projeto.
- Encorajamos você a criar programas de teste para o seu projeto, mesmo que esse trabalho **não precise ser submetido e não será avaliado**. Isso vai te dar a oportunidade de testar facilmente seu trabalho e o dos seus colegas. Você vai achar esses testes especialmente úteis durante sua defesa. De fato, durante a defesa, você é livre para usar seus testes e/ou os testes do colega que está avaliando.
- Submeta seu trabalho no repositório Git designado. Somente o trabalho no repositório Git será avaliado. Se o Deepthought for designado para avaliar seu trabalho, isso ocorrerá após as avaliações entre pares. Se ocorrer um erro em qualquer seção do seu trabalho durante a avaliação do Deepthought, a avaliação será interrompida.

CAPÍTULO III

Instruções sobre IA

● Contexto

Durante sua jornada de aprendizado, a IA pode ajudar em várias tarefas diferentes. Reserve um tempo para explorar as diversas capacidades das ferramentas de IA e como elas podem apoiar seu trabalho. No entanto, sempre as aborde com cautela e avalie criticamente os resultados. Seja código, documentação, ideias ou explicações técnicas, você nunca pode ter certeza absoluta de que sua pergunta foi bem formulada ou que o conteúdo gerado é preciso. Seus colegas são um recurso valioso para ajudá-lo a evitar erros e pontos cegos.

● Mensagem principal

- Use IA para reduzir tarefas repetitivas ou tediosas.
- Desenvolva habilidades de prompting — tanto para código quanto fora dele — que vão beneficiar sua carreira futura.
- Aprenda como sistemas de IA funcionam para melhor antecipar e evitar riscos, vieses e questões éticas comuns.
- Continue construindo habilidades técnicas e interpessoais trabalhando com seus colegas.
- Use apenas conteúdo gerado por IA que você compreenda totalmente e pelo qual possa se responsabilizar.

● Regras para o learner

- Você deve reservar um tempo para explorar ferramentas de IA e entender como elas funcionam, para poder usá-las eticamente e reduzir possíveis vieses.
- Você deve refletir sobre seu problema antes de fazer o prompt — isso ajuda você a escrever prompts mais claros, detalhados e relevantes, usando vocabulário preciso.
- Você deve desenvolver o hábito de checar, revisar, questionar e testar sistematicamente qualquer coisa gerada por IA.
- Você deve sempre buscar revisão por pares — não confie apenas na sua própria validação.

● Resultados esperados da fase

- Desenvolver habilidades de prompting tanto genéricas quanto específicas do domínio.
- Aumentar sua produtividade com o uso eficaz de ferramentas de IA.
- Continuar fortalecendo o pensamento computacional, resolução de problemas, adaptabilidade e colaboração.

● Comentários e exemplos

- Você vai regularmente encontrar situações — provas, avaliações e mais — onde precisará demonstrar compreensão real. Esteja preparado, continue construindo tanto suas habilidades técnicas quanto interpessoais.
- Explicar seu raciocínio e debater com colegas frequentemente revela lacunas na sua compreensão. Faça do aprendizado entre pares uma prioridade.
- Ferramentas de IA frequentemente não têm seu contexto específico e tendem a dar respostas genéricas. Seus colegas, que compartilham seu ambiente, podem oferecer insights mais relevantes e precisos.
- Enquanto a IA tende a gerar a resposta mais provável, seus colegas podem fornecer perspectivas alternativas e nuances valiosas. Conte com eles como um ponto de checagem de qualidade.

✓ Boa prática:

Eu pergunto à IA: "Como testo uma função de ordenação?" Ela me dá algumas ideias. Eu as experimento e reviso os resultados com um colega. Refinamos a abordagem juntos.

✗ Má prática:

Peço à IA para escrever uma função inteira, copio e colo no meu projeto. Durante a avaliação entre pares, não consigo explicar o que ela faz ou por quê. Perco credibilidade — e sou reprovado no projeto.

✓ Boa prática:

Uso IA para ajudar a projetar um parser. Depois percorro a lógica com um colega. Encontramos dois bugs e a reescrevemos juntos — melhor, mais limpa e totalmente compreendida.

✗ Má prática:

Deixo o Copilot gerar meu código para uma parte-chave do projeto. Ele compila, mas não consigo explicar como lida com pipes. Durante a avaliação, não consigo justificar e sou reprovado no projeto.

CAPÍTULO IV

Introdução

O projeto **Push swap** é um projeto de algoritmo muito simples e direto: dados precisam ser ordenados.

Você tem à sua disposição um conjunto de valores inteiros, 2 stacks, e um conjunto de operações para manipular ambas as stacks.

Seu objetivo? Escrever um programa em C chamado `push_swap` que calcule e exiba na saída padrão o menor programa possível, feito de operações da *linguagem Push swap*, que ordene os inteiros recebidos como argumentos.

Fácil?

Vamos ver...

CAPÍTULO V

Objetivos

O objetivo deste projeto é fazer você descobrir a complexidade algorítmica de forma bem concreta.

Ordenar números é fácil; ordená-los *rápido*, com apenas duas stacks e um punhado de movimentos permitidos, é outra história. Ordenar uma lista totalmente aleatória e ordenar uma lista quase ordenada também são duas coisas extremamente diferentes.

Você vai perceber rapidamente como a escolha do algoritmo pode fazer a diferença entre uma vitória rápida e uma rolagem infinita de operações.

CAPÍTULO VI

Parte Obrigatória

VI.1 Requisitos do projeto em grupo

- Este projeto deve ser realizado por exatamente 2 learners trabalhando juntos.
- Ambos os learners devem contribuir de forma significativa para o projeto e entender todos os algoritmos implementados.
- O repositório deve indicar claramente as contribuições de ambos os learners no arquivo README.md.
- Durante a defesa, ambos os learners devem estar presentes e ser capazes de explicar qualquer parte do código.
- A submissão do projeto deve incluir os logins de ambos os learners no repositório.

VI.2 As regras

- Você tem 2 stacks chamadas a e b.
- No início: a stack a contém uma quantidade aleatória de números negativos e/ou positivos, sem duplicatas; a stack b está vazia.
- O objetivo é ordenar em ordem crescente os números na stack a.

sa	Troca os dois primeiros elementos do topo da stack a. Não faz nada se houver apenas um ou nenhum elemento.
sb	Troca os dois primeiros elementos do topo da stack b. Não faz nada se houver apenas um ou nenhum elemento.
ss	sa e sb ao mesmo tempo.
pa	Pega o primeiro elemento do topo de b e o coloca no topo de a. Não faz nada se b estiver vazia.
pb	Pega o primeiro elemento do topo de a e o coloca no topo de b. Não faz nada se a estiver vazia.
ra	Desloca todos os elementos da stack a para cima em uma posição. O primeiro elemento se torna o último.
rb	Desloca todos os elementos da stack b para cima em uma posição. O primeiro elemento se torna o último.
rr	ra e rb ao mesmo tempo.
rra	Desloca todos os elementos da stack a para baixo em uma posição. O último elemento se torna o primeiro.

rrb	Desloca todos os elementos da stack b para baixo em uma posição. O último elemento se torna o primeiro.
rrr	rra e rrb ao mesmo tempo.

VI.3 Requisitos do algoritmo

Para reforçar uma compreensão rigorosa da complexidade algorítmica (*tempo* e *espaço*), você **deve implementar quatro estratégias de ordenação distintas** e integrá-las ao seu programa push_swap. Seu programa deve ser capaz de selecionar uma estratégia em tempo de execução, com base na configuração da entrada.

VI.3.1 Modelo de complexidade e restrições das operações

- Seus algoritmos em C analisam a entrada e geram a sequência apropriada de operações: sa, sb, ss, pa, pb, ra, rb, rr, rra, rrb, rrr.
- O output da sua estratégia é a sequência dessas operações que ordena a stack.
- Quando você declarar uma classe de complexidade, ela deve refletir o custo **medido em número de operações do Push_swap geradas**, não a complexidade teórica de um algoritmo clássico baseado em array.

VI.3.2 Métrica de disorder (obrigatória)

Neste subject, **disorder** é um número entre 0 e 1 que indica o quão longe sua stack a inicial está de estar ordenada. Se os números já estão na ordem correta, o disorder é 0. Se estão na pior ordem possível, o disorder é 1.

Para calculá-lo, pense em olhar para todos os pares possíveis de números na stack. Toda vez que um número maior aparece antes de um menor, esse par conta como um *erro* (mistake).

```
função compute_disorder(stack a):  
    erros = 0  
    total_pares = 0  
    para i de 0 até tamanho(a)-1:  
        para j de i+1 até tamanho(a)-1:  
            total_pares += 1  
            se a[i] > a[j]:  
                erros += 1  
    retorna erros / total_pares
```

Você deve medir o disorder **antes** de fazer qualquer movimento.

VI.3.3 Estratégias exigidas

1. Algoritmo simples ($O(n^2)$)

Implemente pelo menos um algoritmo básico da classe $O(n^2)$. Exemplos: adaptação de insertion sort, selection sort, bubble sort, ou métodos simples de extração de mín/máx.

2. Algoritmo médio ($O(n\sqrt{n})$)

Implemente pelo menos um algoritmo da classe $O(n\sqrt{n})$. Exemplos: ordenação baseada em chunks, particionamento em blocos, bucket sort com $\sqrt{n}$ buckets, ou estratégias baseadas em faixas de valores.

3. Algoritmo complexo ($O(n \log n)$)

Implemente pelo menos um algoritmo da classe $O(n \log n)$. Exemplos: radix sort (LSD ou MSD), merge sort com duas stacks, quick sort com particionamento em stack, heap sort, ou árvore indexada binária.

4. Algoritmo adaptativo customizado (design do learner)

Projete uma estratégia **adaptativa** que selecione métodos internos diferentes dependendo do disorder medido. Você não está restrito a nenhum algoritmo nomeado específico. Seu design deve respeitar as seguintes metas de complexidade por regime:

Disorder baixo	$\text{disorder} < 0.2 \rightarrow O(n^2)$
Disorder médio	$0.2 \leq \text{disorder} < 0.5 \rightarrow O(n\sqrt{n})$
Disorder alto	$\text{disorder} \geq 0.5 \rightarrow O(n \log n)$

Você deve documentar no seu repositório (ex: README.md) a justificativa para seus limiares, as técnicas internas usadas em cada regime, e um breve argumento de complexidade (limites superiores) para tempo e espaço dentro do modelo Push_swap.

VI.4 Exemplo

Para ilustrar o efeito de algumas dessas operações, vamos ordenar uma lista aleatória de inteiros. Neste exemplo, consideraremos que ambas as stacks crescem pela direita.

```
Init a and b:      2 1 3 6 5 8   |  _
Exec sa:           1 2 3 6 5 8   |  _
Exec pb pb pb:     a: 5 3        |  b: 6 3 / 5 2 / 8 1 (topo->fundo)
Exec ra rb (= rr)
Exec rra rrb (= rrr)
Exec sa
Exec pa pa pa:     resultado final ordenado: 1 2 3 5 6 8
```

Os inteiros na stack a ficam ordenados em 12 operações. Consegue fazer melhor?

VI.5 O programa "push_swap"

Nome do Programa	push_swap
Arquivos a Submeter	Makefile, *.h, *.c
Makefile	NAME, all, clean, fclean, re
Argumentos	stack a: uma lista de inteiros
Funções Externas	read, write, malloc, free, exit · ft_printf ou equivalente codado por VOCÊ
Libft autorizada	Sim
Descrição	Ordenar stacks

Seu projeto deve cumprir as seguintes regras:

- Você precisa entregar um Makefile que compilará seus arquivos fonte. Ele não deve fazer relink.
- Variáveis globais são proibidas.
- O programa recebe a stack a (primeiro argumento = topo) e um seletor de estratégia opcional: --simple, --medium, --complex ou --adaptive (padrão se nenhum for informado).
- O programa deve exibir a menor lista possível de operações para ordenar a stack a, separadas por \n e nada mais.
- A seleção de estratégia deve funcionar para todas as entradas válidas, independentemente do tamanho ou disorder.
- Se nenhum parâmetro for especificado, o programa não deve exibir nada.
- Em caso de erro (argumentos não inteiros, fora da faixa, duplicados), exibir "Error" seguido de \n na saída de erro padrão.
- O binário deve embutir as quatro estratégias; nome e complexidade devem estar disponíveis no modo --bench.
- O modo --bench (stderr) deve exibir: disorder (%), estratégia usada + complexidade, total de operações, e a contagem de cada tipo de operação.

VI.5.1 Exemplos de uso

```
$> ./push_swap 2 1 3 6 5 8
ra
pb
rra
pb
pb
ra
pb
ra
pb
pb
pa
pa
pa
pa
pa
pa
```

Seleção padrão (--adaptive) e contagem de operações:

```
$> ARG="4 67 3 87 23"; ./push_swap --adaptive $ARG | wc -l
13
```

Forçar a estratégia simples ($O(n^2)$):

```
$> ./push_swap --simple 5 4 3 2 1
rra
pb
rra
pb
rra
pb
ra
pb
pb
pa
pa
pa
pa
pa
pa
```

Forçar a estratégia complexa e verificar com o checker:

```
$> ARG="4 67 3 87 23"; ./push_swap --complex $ARG | ./checker_linux $ARG
OK
```

Rodar com benchmark ativado (métricas em stderr):

```
$> shuf -i 0-9999 -n 500 > args.txt ; ./push_swap --bench $(cat args.txt) \  
2> bench.txt | ./checker_linux $(cat args.txt)  
OK  
$> cat bench.txt  
[bench] disorder: 49.93%  
[bench] strategy: Adaptive / O(n2)  
[bench] total_ops: 7997  
[bench] sa: 0 sb: 0 ss: 0 pa: 500 pb: 500  
[bench] ra: 4840 rb: 1098 rr: 0 rra: 0 rrb: 1059 rrr: 0
```

Exemplos de tratamento de erro:

```
$> ./push_swap --adaptive 0 one 2 3  
Error  
$> ./push_swap --simple 3 2 3  
Error
```


VI.6 Benchmark de Performance

Para validar este projeto, você deve atingir certas metas de performance com um número mínimo de operações:

100 números	< 2000 (mínimo) · < 1500 (bom) · < 700 (excelente)
500 números	< 12000 (mínimo) · < 8000 (bom) · < 5500 (excelente)

Tudo isso será verificado durante sua avaliação usando o checker fornecido.

CAPÍTULO VII

Requisitos do Readme

Um arquivo README.md deve ser fornecido na raiz do seu repositório Git. Seu propósito é permitir que qualquer pessoa não familiarizada com o projeto (colegas, staff, recrutadores, etc.) entenda rapidamente do que se trata o projeto, como executá-lo, e onde encontrar mais informações sobre o tema.

O README.md deve incluir, no mínimo:

- A primeira linha deve estar em itálico e dizer (em inglês, texto literal exigido): *"This project has been created as part of the 42 curriculum by <login1>[, <login2>[, <login3>[...]]]."*
- Uma seção **"Description"** apresentando claramente o projeto, objetivo e visão geral.
- Uma seção **"Instructions"** com informações sobre compilação, instalação e/ou execução.
- Uma seção **"Resources"** listando referências clássicas relacionadas ao tema, além de uma descrição de como a IA foi usada — especificando para quais tarefas e em quais partes do projeto.

Seções adicionais podem ser exigidas dependendo do projeto (ex: exemplos de uso, lista de funcionalidades, escolhas técnicas, etc.).

Uma explicação detalhada e justificativa dos algoritmos escolhidos para este projeto também deve ser incluída.

Inglês é recomendado; alternativamente, você pode usar o idioma principal do seu campus.

CAPÍTULO VIII

Parte Bônus

Devido à sua simplicidade, este projeto oferece oportunidades limitadas para funcionalidades adicionais. No entanto, por que não criar seu próprio checker?

Graças ao programa checker, você será capaz de verificar se a lista de operações gerada pelo programa push_swap realmente ordena a stack corretamente.

A parte bônus só será avaliada se a parte obrigatória estiver perfeita. Perfeita significa que a parte obrigatória foi totalmente concluída e funciona sem erros — incluindo validar todos os benchmarks sem exceção. Se você não passou em TODOS os requisitos obrigatórios, sua parte bônus não será avaliada de forma alguma.

VIII.1 O programa "checker"

Nome do Programa	checker
Arquivos a Submeter	*.h, *.c
Makefile	bonus
Argumentos	stack a: uma lista de inteiros
Funções Externas	read, write, malloc, free, exit · ft_printf ou equivalente codado por VOCÊ
Libft autorizada	Sim
Descrição	Executar as operações de ordenação

- Recebe a stack a como argumento (primeiro argumento = topo). Sem argumento, para e não exibe nada.
- Lê operações da entrada padrão, uma por linha (\n). Após ler tudo, executa as operações na stack recebida.
- Se a stack a estiver ordenada e a stack b vazia → "OK\n" em stdout.
- Caso contrário → "KO\n" em stdout.
- Erros (argumentos inválidos, duplicados, operação inexistente ou mal formatada) → "Error\n" em stderr.

```
$>./checker 3 2 1 0
rra
pb
sa
rra
pa
OK

$>./checker 3 2 1 0
sa
rra
pb
KO

$>./checker 3 2 one 0
Error

$>./checker "" 1
Error
```

Você NÃO precisa reproduzir exatamente o mesmo comportamento do binário fornecido. É obrigatório tratar erros, mas cabe a você decidir como quer parsear os argumentos.

CAPÍTULO IX

Submissão e avaliação entre pares

Submeta sua atividade no seu repositório Git como de costume. Somente o trabalho dentro do seu repositório será avaliado durante a defesa. Não hesite em checar novamente os nomes dos seus arquivos para garantir que estão corretos.

Requisitos de submissão para projeto em grupo:

- Ambos os learners devem estar listados como contribuidores no repositório.
- O README.md deve documentar claramente as contribuições de ambos os learners.
- Ambos os learners devem estar presentes durante a defesa de avaliação entre pares.
- Cada learner deve ser capaz de explicar e defender qualquer parte do código.

Durante a avaliação, uma breve **modificação do projeto** pode ocasionalmente ser solicitada. Isso pode envolver uma pequena mudança de comportamento, algumas linhas de código para escrever ou reescrever, ou uma funcionalidade fácil de adicionar.

Embora essa etapa possa não se aplicar a todo projeto, você deve estar preparado para ela caso seja mencionada nas diretrizes de avaliação.

Essa etapa serve para verificar sua compreensão real de uma parte específica do projeto. A modificação pode ser realizada em qualquer ambiente de desenvolvimento que você escolher, e deve ser viável em poucos minutos — a menos que um prazo específico seja definido como parte da avaliação.

Os detalhes (escopo, alvo, etc.) serão especificados nas diretrizes de avaliação e podem variar de uma avaliação para outra, mesmo para o mesmo projeto.